

structure_hk.md — 遺傳蜂群商業操作系統

版本 2.1 · 研究整合版 · 2026 年 7 月
實作對應：§12 · SDD 規格包：[planning/structure/](#) · 現況：[status.md](#)
(英文原文：[structure.md](#))

0. 這是甚麼

這是一個受治理、可自我改進的多代理系統，其能力包括：

- 1. 學習一家企業實際如何運作（來自文件、專家，以及真實事件日誌）。
- 2. 把這些知識提煉成可重用的規則、技能、工作流程與作業手冊。
- 3. 透過有邊界且可審計的代理工作流程來執行工作。
- 4. 在沙盒中演化這些工作流程，而不是直接在生產環境中修改。

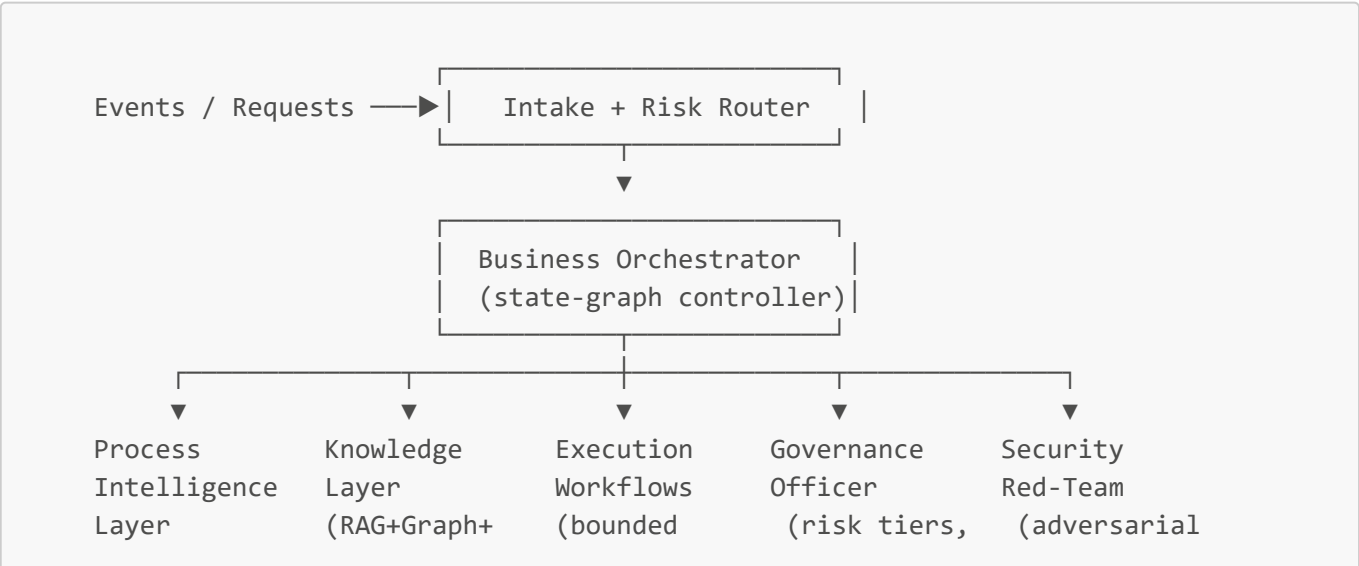
設計優先次序依次為：安全性 → 可審計性 → 正確性 → 效率 → 自主性。自主性不是預設授予，而是要按每個工作流程以證據逐步取得。

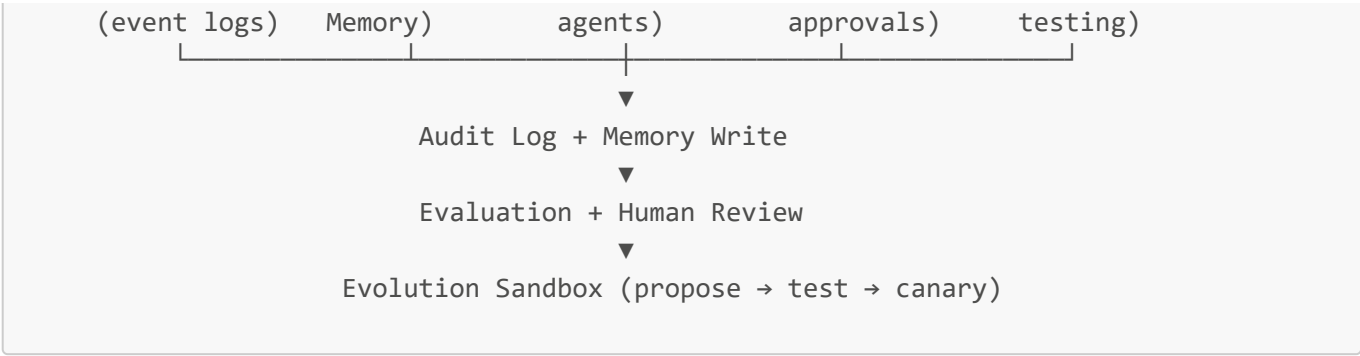
本文件是架構願景與權威來源 (**source of truth**) 。可執行的分解 (requirements / design / tasks) 、產品標竿證據，以及 as-built 說明見 §12 與相關 planning 檔案——它們細化本文件，並不取代本文件。

1. 核心原則

- 1. 證據優先於意見。從真實軌跡中學習，而不只依賴人們口頭描述他們如何工作。
- 2. 有邊界的自主性。每個行動都必須有風險等級、權限範圍，以及在需要時的人類關卡。
- 3. 一切都必須可測試。任何代理、提示或工作流程，在通過評估前都不得進入生產環境。
- 4. 沙盒演化。演化引擎只負責提出方案，絕不直接改動生產環境。
- 5. 全程保留來源。每條規則、每個決策及每段記憶，都必須可追溯到來源。
- 6. 可逆性優先。優先採用可逆操作；其餘操作則必須具備回滾方案。
- 7. 以人為中心。顯示信心、顯示證據，並讓修正變得容易。

2. 高層架構





六個層級共同運作：流程智能（2.3）、知識（3）、執行（4）、演化（5）、治理（6）及 安全（7），而這一切都由 評估（8）包裹與約束。

2.3 流程智能層

蜂群不應只從文件與訪談中學習，而應從真實營運軌跡中學習：工單、CRM/ERP 操作、行事曆事件、電郵、審批、檔案編輯、API 呼叫，以及完成紀錄。流程探勘會把這些日誌轉化成可發現的工作流程模型、一致性檢查與瓶頸分析，也就是你的「Shadow Mode」的實證版本。

新增代理：

- **Process Miner Agent** — 從事件日誌中發掘真實工作流程。
- **Task Mining Agent** — 在獲准情況下觀察 UI / 人類層面的步驟。
- **Conformance Agent** — 比較實際工作與已記錄的 SOP 是否一致。
- **Bottleneck Analyzer** — 找出延誤、循環、重工與交接失敗。
- **Causal Improvement Agent** — 提出有機會改善結果的干預措施。

事件日誌綱要：

```
event:
  id: "evt_..."
  timestamp: "2026-07-06T14:03:00Z"
  actor_type: "human | agent | system"
  actor_id: "user_or_agent_id"
  process_id: "customer_onboarding"
  case_id: "customer_12345"
  activity: "review_contract"
  input_refs: ["doc_contract_v3"]
  output_refs: ["approval_decision_789"]
  tools_used: ["crm", "email"]
  decision_point: true
  decision_reason_summary: "Contract had non-standard liability clause."
  confidence: 0.82
  risk_tier: "medium"
  human_approved: true
  outcome:
    status: "completed"
    latency_minutes: 42
    quality_score: 0.94
```

3. 知識擷取、提煉與記憶

3.1 引出方法

專家知識大多屬於隱性知識，例如哪些訊號重要、何時應覆寫規則、何時會感覺「不對勁」。認知工作分析 (Cognitive Task Analysis) 與關鍵決策法 (Critical Decision Method) 是把這些知識顯性化的成熟方法。把單一的「Expert Shadow Agent」升級成多方法擷取系統。

方法	最適用於	輸出
Shadow Mode	真實操作	事件日誌、行動軌跡
Critical Decision Interview	罕見 / 高風險決策	決策需求卡
Think-Aloud Session	例行專家工作	逐步啟發式規則
Exception Interview	邊界個案	例外情況知識庫
Retrospective Review	已完成個案	經驗教訓
Apprentice Mode	專家教導蜂群	技能、作業手冊

3.2 決策需求卡

```
decision_requirement:
  id: "drc_contract_exception_001"
  domain: "legal_operations"
  decision_point: "approve_non_standard_clause"
  expert_sources: ["senior_counsel_A", "contract_manager_B"]
  context_signals: ["customer_size", "liability_cap", "jurisdiction",
"renewal_value"]
  cues_experts_notice:
    - "Clause shifts uncapped indirect damages to company."
    - "Customer insists on governing law outside approved list."
  normal_action: "route_to_legal_review"
  exception_paths:
    - condition: "enterprise_customer AND pre-approved fallback accepted"
      action: "approve_with_note"
  red_flags: ["unlimited liability", "data protection indemnity"]
  required_evidence: ["contract_diff", "customer_risk_profile",
"approval_history"]
  risk_tier: "high"
  human_approval_required: true
  validation_tests:
    - "Does recommendation match senior counsel decision on historical cases?"
  confidence: 0.78
  last_reviewed: "2026-07-06"
```

3.3 混合式記憶

單一泛用的「知識庫」並不足夠。長時間運行的代理需要分層記憶，包括原始觀察、更高層次的反思，以及可檢索的長期儲存，這種模式已在 Generative Agents 及階層式記憶代理設計中得到驗證。

記憶類型	儲存內容	範例
Event	原始營運日誌	"Agent sent invoice at 9:42 AM."
Episodic	個案敘事	"This renewal almost failed — legal was pulled in late."
Semantic	事實 / 規則	"Enterprise contracts over 250k need legal review."
Procedural	技能 / 工作流程	"How to onboard a new client."
Decision	決策與原因	"We approved exception X because Y."
Exception	邊界個案	"If supplier in region Z, use alternate process."
Evaluation	測試結果	"Workflow v12 failed privacy test."
Provenance	來源歸屬	"Rule came from SOP v4 and expert Alice."

3.4 檢索：分層混合式 (以 LightRAG 為核心)

這裡刻意避免採用 GraphRAG 式的社群摘要，因為它對每個分塊都要做抽取，再生成社群報告，令初始索引及更重要的每次新文件進來後重建索引的成本都高得難以接受，特別不適合持續接收事件日誌與文件的系統。

取而代之的是按成本分層的檢索堆疊。大多數查詢根本不需要進入昂貴層級。

第 0 層 — 向量搜尋 (預設、最便宜)。對分塊文件做語意相似度搜尋，可處理大部分「找出相關段落」的查詢。無需圖結構，也不需額外 LLM 呼叫。

第 1 層 — LightRAG 圖層 (關聯推理)。

- 以圖為基礎的文字索引，支援雙層檢索：低層 (實體特定) + 高層 (主題性)。
- **增量更新** — 新文件 / 事件可直接加入，無需重建整個圖。這是選用 LightRAG 的主要原因。
- 適合配合自託管 / 本地模型運行，以進一步降低 token 成本。
- 可回答關聯性問題，例如："which obligations depend on this contract?" 以及 "who touched this case and in what order?"

第 2 層 — 階層式摘要 (RAPTOR 風格，可選，按需建立)。以遞迴方式進行分群與摘要，只為那些經常出現全語料問題的語料建立，例如「失敗 onboarding 個案中反覆出現的根本原因」。採延遲建立，而不是為整個知識庫預先建立。

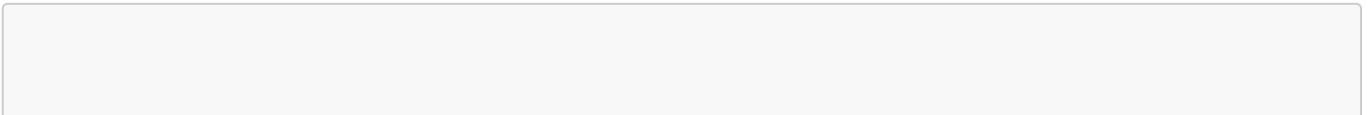
常駐層 — 來源層。無論由哪一層提供答案，每次回應都必須引用其來源文件、專家、事件日誌或決策。

升級規則：先從第 0 層開始；只有在查詢需要關聯 / 多跳推理時才升級至第 1 層；只有在需要全域綜合時才升級至第 2 層。這樣可讓 80% 以上流量停留在最便宜的層級。

現成方案：如果不想自行建置，AnythingLLM 或 RAGFlow 都提供開源、適合本地模型的 RAG 平台；LightRAG 可作為其後方的圖層整合進去。

評估：應分開評分檢索的上下文相關性、答案相關性與忠實度，因為薄弱的檢索器會默默污染每一個代理。

3.5 資料夾結構



```
business/
├── process-intelligence/
│   ├── event-logs/
│   ├── discovered-processes/
│   ├── conformance-reports/
│   ├── bottlenecks/
│   └── causal-hypotheses/
├── knowledge-base/
│   ├── rules/
│   ├── decision-patterns/
│   ├── exceptions/
│   ├── best-practices/
│   ├── tacit-knowledge/
│   └── provenance/
├── experts/
│   ├── profiles/
│   ├── shadow-logs/
│   ├── decision-requirement-cards/
│   └── interview-transcripts/
├── materials/
│   ├── documents/
│   ├── regulations/
│   └── sops/
├── distilled/
│   ├── skills/
│   ├── prompts/
│   ├── workflows/
│   ├── checklists/
│   └── playbooks/
├── memory/
│   ├── episodic/
│   ├── semantic/
│   ├── procedural/
│   ├── decision-memory/
│   └── evaluation-memory/
├── evals/
│   ├── golden-tasks/
│   ├── regression-tests/
│   ├── adversarial-tests/
│   ├── human-review-sets/
│   └── benchmark-results/
├── governance/
│   ├── ai-inventory/
│   ├── use-case-risk-tiering/
│   ├── risk-assessments/
│   ├── human-approval-policy/
│   ├── audit-logs/
│   ├── model-cards/
│   └── assurance-cases/
├── security/
│   ├── threat-models/
│   ├── tool-permissions/
│   └── prompt-injection-tests/
```

```
|   |— red-team-results/
|   |— incident-reports/
|— evolution/
|   |— workflow-dna/
|   |— successful-variants/
|   |— failed-experiments/
|   |— mutation-history/
|   |— lessons-learned/
```

4. 執行：Workflow DNA

4.1 綱要

```
workflow_dna:
  id: "wf_customer_onboarding_v12"
  name: "Customer Onboarding"
  domain: "operations"
  objective: "Onboard customer with minimal delay and compliance risk."
  owner: "business_orchestrator"
  version: "12.0"
  inputs: ["signed_contract", "customer_profile", "billing_details"]
  preconditions:
    - "contract_status == signed"
    - "customer_risk_score <= threshold OR legal_approval == true"
  steps:
    - id: "verify_contract"
      agent: "quality_compliance_agent"
      tools: ["contract_parser", "policy_retriever"]
    - id: "create_customer_record"
      agent: "execution_agent"
      tools: ["crm"]
    - id: "configure_billing"
      agent: "finance_ops_agent"
      tools: ["billing_system"]
    - id: "send_welcome_packet"
      agent: "communications_agent"
      tools: ["email"]
  memory_reads: ["contract_rules", "customer_exceptions", "past_failures"]
  memory_writes: ["event_log", "decision_memory", "lessons_learned"]
  guardrails:
    human_approval_required_if:
      - "risk_tier == high"
      - "contract_exception_detected == true"
      - "tool_action_is_irreversible == true"
  verification:
    required_checks:
      - "crm_record_created"
      - "billing_config_validated"
      - "welcome_packet_sent"
      - "audit_log_complete"
```

```

rollback:
  reversible: true
  rollback_steps: ["disable_customer_record", "void_initial_invoice",
"notify_ops_owner"]
  fitness_metrics:
    - "cycle_time"
    - "error_rate"
    - "customer_satisfaction"
    - "compliance_pass_rate"
    - "human_escalation_rate"
    - "cost_per_case"

```

4.2 執行模式

應使用**有邊界的狀態圖**，而不是自由形態的蜂群。推理 / 行動迴圈 (ReAct 風格，即思考、行動與觀察交錯進行) 適合放在節點內部，但整個圖本身必須負責強制狀態、權限與人類在迴圈中的審批關卡。

```

Event → Intake Router → Risk Classifier → Orchestrator
    → [Research] → [Execution] → [Verification] → [Compliance] → [Human Gate]
    → Audit Log + Memory Write → Evaluation → Evolution Sandbox

```

5. 演化引擎 (沙盒化)

5.1 唯一不可談判的規則

Evolution Manager 絕不能直接改動生產環境。它只能：提出變體 → 在沙盒中測試 → 與基線比較 → 請求批准 → 金絲雀部署 → 失敗時自動回滾。

這一條約束，足以把一個高風險的自主蜂群系統轉化成受控且可審計的系統。

5.2 適應度函數

不要根據主觀偏好進行演化。每個變體都要評分：

$$[F = w_q Q + w_s S + w_c C + w_e E + w_h H - w_r R - w_l L - w_k K]$$

其中 (Q)=品質，(S)=安全，(C)=合規，(E)=效率，(H)=人類滿意度，(R)=風險懲罰，(L)=延遲懲罰，(K)=成本懲罰。若目標互相衝突，應使用 **Pareto selection**，而不是把所有目標壓縮成單一標量。

5.3 流程管道

1. Observe production / shadow traces
2. Detect failures, bottlenecks, or opportunities
3. Generate variants (prompt / workflow / tool-use / role / expert-pattern crossover)
4. Test offline against golden tasks
5. Run security + adversarial tests
6. Run compliance checks

7. Replay on historical cases (simulation)
 8. Human review if risk tier requires
 9. Canary deploy to small scope
 10. Monitor metrics
 11. Promote / rollback / retire
 12. Store lessons in evolution memory

自然語言反思在這裡是一種很強的優化器。反思式提示演化方法（例如 GEPA）表明，只需少量以語言診斷的軌跡，就可以勝過多輪基於標量回報的 RL；這點特別適合資料稀缺的商業場景。但這些迴圈必須始終維持在上述沙盒關卡之內。

5.4 晉升規則

只有在以下條件全部滿足時，某個變體才可晉升：(1) 改善目標指標、(2) 不使安全或合規倒退、(3) 通過回歸與對抗測試、(4) 具備回滾方案、(5) 擁有完整審計日誌，以及 (6) 在風險等級要求時已獲得人工簽核。

6. 治理、風險與合規

應以既有框架為基礎，而不是自行發明一套。

- **NIST AI RMF (AI 100-1)** — 值得信賴 AI 的 map/measure/manage 風險骨幹框架。
- **ISO/IEC 42001** — 管理系統層的外框。這是全球首個 AI 管理系統標準，圍繞 Plan-Do-Check-Act 方法建立，用於建立、實施、維護及持續改進 AI 管理系統[1]，涵蓋風險管理、AI 系統影響評估、系統生命週期管理，以及第三方供應商監督[3]。
- **EU AI Act** — 如果你的系統涉及歐盟用戶、員工或受規管決策，便會適用。該法案於 2024 年 8 月 1 日生效；被禁止的做法與 AI 素養義務自 2025 年 2 月 2 日起適用，而 GPAI 模型義務則自 2025 年 8 月 2 日起生效[5]。需要留意的是，時間表仍在變動：附錄 III 的高風險義務正透過 Digital Omnibus 由 2026 年 8 月 2 日延後至 2027 年 12 月 2 日[9]，但這項變更只有在正式通過並公布後才具有法律效力。這與本系統直接相關，因為法案把用於就業相關決策的 AI，例如招聘、候選人篩選、績效評估、任務分配、員工監察，以及晉升或終止僱用，分類為高風險[2]。如果蜂群系統觸及這些領域，便應預期必須符合風險管理、數據治理、技術文件、紀錄保存、透明度、人類監督、準確性、穩健性與網絡安全，以及上市後監察與事故通報等要求[7]。

6.1 自主性風險分級

等級	自主性	允許行為
0	觀察	只記錄與摘要。
1	建議	提出建議；由人類執行。
2	草擬	準備產出物；發送或執行前由人類批准。
3	執行（可逆）	如存在回滾方案且風險低，可自行行動。
4	執行 + 關卡	可行動，但關鍵步驟須經人類批准。
5	受限制	在建立 assurance case 前，不得自主行動。

6.2 必備產出物

AI inventory · use-case risk tiering · human-approval policy · audit logs · incident-response plan · rollback plans · data-retention policy · vendor/model register · tool-permission register · assurance cases · model cards.

7. 安全

代理式系統把攻擊面擴大到遠超傳統 AppSec。現時有兩套 OWASP 參考同時適用：模型層對應**Top 10 for LLM Applications (2025)**，自主層對應 **Top 10 for Agentic Applications (2026)**。後者是一套經同行評審的框架，用來識別能夠在工作流程中規劃、行動及作決策的自主 AI 系統最關鍵的風險[3]，其重點威脅包括 Agent Behavior Hijacking、Tool Misuse and Exploitation，以及 Identity and Privilege Abuse[7]。

對蜂群系統而言，最重要的設計事實是：間接提示注入是大多數代理式系統的關鍵威脅，而即使經過對齊與過濾，也應假設提示注入仍然可能發生，因此控制爆炸半徑至關重要[8]。同樣地，系統提示本身不是安全控制；如果機密已放進提示中，它其實已經外洩[7]。安全必須在 LLM 之外以確定性方式執行。

7.1 控制矩陣（對應 OWASP LLM 2025）

風險	OWASP	控制措施
Prompt injection (尤其是間接)	LLM01	把所有檢索內容 / 用戶內容都視為不可信；分離指令與資料；限制爆炸半徑。
敏感資訊洩露	LLM02	對輸出 / 日誌做 DLP；提示中絕不存放秘密；設定保留期限。
供應鏈	LLM03	建立模型 / 工具 / adapter 登記冊；保留來源；執行依賴與 SBOM 掃描。
數據與模型投毒	LLM04	審核 fine-tunes / LoRAs；驗證檢索來源；對記憶寫入做過濾。
不當輸出處理	LLM05	把模型輸出當作不可信輸入；任何執行前都先消毒。
過度代理權限	LLM06	依風險分級授予自主性；採最小權限；設置審批關卡。
系統提示洩露	LLM07	提示中不要放秘密 / 角色設定；在外部執行授權控制。
向量 / 嵌入弱點	LLM08	對向量儲存做租戶隔離；偵測被投毒內容。
錯誤資訊	LLM09	做 grounding 與引用來源；顯示信心；高風險情境下安排人工審核。
無界資源消耗	LLM10	設定速率限制、逾時與預算上限（「denial-of-wallet」防禦）。

就過度代理權限而言，OWASP 的實務指引與 §6.1 的風險分級高度一致：限制 LLM 可存取的工具、要求敏感或不可逆行動必須有人類批准，並對數據、API 與工具採用最小權限原則[6]。

7.2 額外的代理式控制

- **Tool Permission Broker** — 針對每項任務發放狹窄、臨時且具範圍限制的憑證。
- **Memory-poisoning defense** — 對高影響記憶寫入採用來源追蹤 + 人工審核（這已是一類被明確命名的代理式風險）。
- **Skill/plugin vetting** — 第三方代理技能是現實存在的供應鏈攻擊面；必須掃描並固定版本。
- **Full observability** — 對模型呼叫、工具呼叫與代理間通訊維持單一審計軌跡。
- **AI incident response** — 為 GenAI 特有事故建立明確的處置 runbook。

8. 評估系統

這是大多數「蜂群」設計中最大的缺口。每個代理、技能、工作流程與提示都必須具備：

1. Golden task set 2. Regression tests 3. Adversarial tests 4. Human-review set
2. Historical-replay set 6. Cost/latency benchmark 7. Business-outcome metric 8. Safety/compliance score

評估必須在真實、多步驟、會使用工具的環境中進行。像 AgentBench 與 SWE-bench 這類代理基準所帶來的教訓是：孤立的提示測試，並不能預測真實任務表現。

評估卡：

```
evaluation:
  target: "wf_customer_onboarding_v12"
  eval_type: "workflow_regression"
  test_set: "historical_onboarding_cases_q2"
  metrics:
    quality_score: 0.94
    compliance_pass_rate: 0.99
    average_cycle_time_minutes: 38
    escalation_rate: 0.12
    hallucination_rate: 0.01
    unauthorized_tool_attempts: 0
    cost_per_case_usd: 0.42
  result: "pass"
  promotion_decision: "canary_only"
  reviewer: "ops_lead"
```

9. 代理名冊

控制 / 中樞

代理	用途
Business Orchestrator	路由工作、管理狀態，並持有整體目標。
Evolution Manager	提出並測試變體（只限沙盒）。
Evaluation Harness	執行 golden／regression／adversarial／replay 測試套件。
Governance Officer	套用風險分級、批准規則與審計要求。
Security Red-Team	測試提示注入、工具濫用、洩露與不安全自主行為。
Memory Steward	維護記憶品質、來源追蹤與失效機制。
Tool Permission Broker	授予具範圍限制且臨時的工具存取權。
Incident Commander	處理失敗、回滾與事後檢討。

學習

代理	用途
Expert Shadow	在取得許可下觀察專家。
Cognitive Task Analyst	把訪談轉換成決策卡與啟發式規則。
Process Miner	從日誌中發掘工作流程。
Knowledge Distiller	把原始材料轉化成規則、技能與作業手冊。
Knowledge Curator	驗證、去重與整理知識。

10. 人機互動規則

綜合 20 多年的指引 (Microsoft 的 Guidelines for Human-AI Interaction)，蜂群系統必須：顯示信心與不確定性；解釋其所使用的證據；在執行前預覽將採取的行動；讓修正只需一下點擊；允許覆寫；把被拒絕的建議儲存為訓練資料；在上下文不足時主動要求澄清；而且絕不能用自信語氣掩飾不確定性。

11. 90 日推行計劃

- 第 1–14 日 — 基礎建設：** 資料夾結構 · 事件日誌綱要 · AI inventory · 風險分級 · 審計日誌 · 首 20 個 golden tasks。
- 第 15–30 日 — Shadow Learning：** 啟用 Shadow Mode · 專家訪談 · 收集事件日誌 · 首批決策卡 · 知識擷取管道。
- 第 31–60 日 — 受控 Co-Pilot：** RAG + 來源追蹤 · 審批關卡 · 首個 Workflow DNA · 回歸測試 · 為低 / 中風險工作流程啟用 Co-Pilot Mode。
- 第 61–90 日 — 演化沙盒：** 變體生成 · 提示 / 工作流程變異 · 評估 harness · 金絲雀部署 · 自動回滾 · 只晉升通過安全、品質與商業測試的變體。

11.1 As-built 能力關卡 (產品標竿)

以上 90 日分帶仍是願景時間表。本倉庫交付以能力關卡追蹤 (不是日曆天數)。產品標竿 mark ~100 大致對應：

階段	structure 分帶	As-built (本倉庫)
A 基礎	第 1–14 日	business/ 樹、事件綱要、inventory、風險分級、審計、≥20 golden
B Shadow 學習	第 15–30 日	事件 ingest + PI 產物；DRC 模板與範例卡
C 受控 co-pilot	第 31–60 日	Postgres 控制平面、DNA 執行、人類關卡、Tier-0/1 檢索、FE 營運台
D 演化沙盒	第 61–90 日	Corpus 評估、canary/rollback、自我改進 (reflect/propose)、Improve UI

證據：[status.md](#)、[structure_scorecard_100.md](#)、[mark_100_verification.md](#)、[reviews/e1_operator_checklist.md](#)、[planning/gap_analysis_for_structure.md](#)。

12. 實作對應 (SDD + as-built)

12.1 規格驅動分解

可執行的子功能規格 (不可用其取代本架構文件)：

路徑	內容
planning/structure/README.md	索引、依賴順序、模板
planning/structure/nn_*/requirements.md	EARS 需求
planning/structure/nn_*/design.md	完整設計 (v2.0)
planning/structure/nn_*/tasks.md	實作任務 (v2.0 · 已標記完成)
planning/structure/DESIGN_QUALITY_SCORE.md	設計組合分數
planning/structure/TASKS_QUALITY_SCORE.md	任務組合分數
planning/structure/IMPLEMENTATION_STATUS.md	實作矩陣
planning/gap_analysis_for_structure.md	實作對任務落差分數

12.2 章節 → 子功能規格

structure.md / structure_hk.md	規格資料夾
\$0-1 憲章 / 原則	01_system-charter-and-design-priorities
\$3.5 資料夾結構	02_business-artifact-repository
\$2 Intake + 風險路由	03_intake-and-risk-router
\$6 治理	04_governance-risk-tiers-and-artifacts
\$7 安全	05_security-controls-and-tool-broker
\$2.3 流程智慧	06_process-intelligence-layer
\$3.1-3.2 擷取 + DRC	07_knowledge-elicitation-and-decision-cards
\$3.3 混合記憶	08_hybrid-memory-system
\$3.4 分層檢索	09_tiered-hybrid-retrieval
\$4.1 Workflow DNA	10_workflow-dna-definition
\$4.2 執行模式	11_bounded-workflow-execution
\$4 護欄 + 審計路徑	12_human-gates-and-audit-logging
\$8 評估	13_evaluation-harness-and-corpus

structure.md / structure_hk.md 規格資料夾	
\$5 演化沙盒	14_evolution-sandbox-engine
\$9 代理名冊	15_agent-roster-and-control-roles
\$10 人機規則	16_human-ai-interaction-rules
\$11 推行計劃	17_phased-rollout-and-operator-path

12.3 As-built 實現說明 (不削弱架構)

以下記錄本倉庫今日如何實現架構，並不刪減未來深度。

主題	架構意圖	As-built 實現
Harness	受治理代理環境	雙 harness：Trae (.trae/) + Grok Build (.grok/)，npm run sync
控制平面	API + 持久狀態	FastAPI + Postgres runtime_state JSONB；JSON 檔 = 備份 / 種子
工具適配器	真實執行	本機 adapters + 持久 tool_effects；外部 CRM/email = 稍後
工具中介	範圍化權限	allow-list n DNA tools n RBAC n 關卡；短時 OAuth 每工具 = 稍後
PI 代理	Miner / 合規 / 瓶頸 / 因果	PI 服務 + 磁碟產物 (非五個獨立 LLM agent)
檢索 \$3.4	Tier 0 / 1 / 2	Tier 0 關鍵字+hash embed + 來源；Tier 1 entity multi-hop (LightRAG-lite)；Tier 2 + 完整 LightRAG 產品 = 稍後
知識圖	Agent-native graph	K1-lite 抽取 / 運算子 + 可選 federation 匯出
演化 \$5	僅沙盒	變體 sandbox_only；corpus 評估；canary；版本化晉升；rollback；不改寫 host 程式碼
自我改進	反思迴圈 (GEPA 風格)	Auto-reflect、lessons、auto-propose、Loop runner、FE Improve 管線
DNA 生產安全	關卡 + 回滾	business:validate + runtime activate_workflow_version 生產 DNA 檢查
前端	營運主控台	Next.js live ops (DEMO_MODE=false)；真實表單；Improve；/app/evolution
操作者證明	端到端路徑	E1：login → run → human gate → complete → improve (test_e1_operator_path)

12.4 明確非目標 (目前產品標竿)

mark ~100 不把下列項目視為未完成的 structure 要求：

- 完整商用 LightRAG／Neo4j 生產 mesh
- 外部 live CRM／email／billing SaaS adapters
- DGM 式 host 應用自我改寫
- 常駐伺服器的全量 Playwright UI CI
- 填滿 [business/](#) 每一葉節點的無限企業內容

12.5 Runtime / 營運入口

層	入口
Backend	backend/ — FastAPI、 runtime.py 、 infrastructure/*
Frontend	frontend/ — Next.js 營運主控台
業務語料	business/
延續性	memory/handoff.md 、 memory/project.md 、 status.md

13. 參考資料

- NIST AI RMF 1.0 (AI 100-1); ISO/IEC 42001:2023; EU AI Act + Digital Omnibus (2025–2026).
- OWASP Top 10 for LLM Applications (2025); OWASP Top 10 for Agentic Applications (2026); OWASP Agentic Security Initiative.
- Process Mining Manifesto (IEEE Task Force on Process Mining).
- ReAct (Yao et al.); RAG (Lewis et al.); Generative Agents (Park et al.); GEPA (Agrawal et al.); AgentBench (Liu et al.).
- Microsoft, Guidelines for Human-AI Interaction (Amershi et al., CHI 2019); Cognitive Task Analysis / Critical Decision Method (Hoffman, Crandall, Shadbolt).
- 倉庫內 SDD：[planning/structure/](#)；產品證據：[status.md](#)、[structure_scorecard_100.md](#)、[mark_100_verification.md](#)。
- 英文對照：[structure.md](#)。